



TESINA PROGETTUALE CONCLUSIVA
PERCORSO Cyber Security Analyst
A.A. 2024-2026

SecOps Cyber Range Portal

CANDIDATI
Caccialanza Simone
Fossati Matteo
Abdurrahaman Ahamed Rilah

Indice

Introduzione	4
Capitolo 1 — Contesto, obiettivi e requisiti	5
1.1 I cyber range come strumento didattico	5
1.2 Obiettivi del progetto	5
1.3 Requisiti funzionali e non funzionali	5
Requisiti funzionali	5
Requisiti non funzionali	6
Capitolo 2 — Analisi e scelte progettuali	7
2.1 Analisi dello stato dell'arte	7
2.2 Scelta dello stack tecnologico	7
2.3 Modello dei dati	8
2.4 Metodologia di sviluppo	8
Capitolo 3 — Architettura del sistema	9
3.1 Vista d'insieme	9
3.2 Il livello client	9
3.3 Il livello server	10
3.4 Flusso di avvio dell'applicazione	10
Capitolo 4 — Implementazione dei moduli funzionali	11
4.1 Autenticazione e gestione utenti	11
4.2 Dashboard e Security Posture Score	11
4.3 Gestione degli asset	12
4.4 Gestione delle vulnerabilità (CVE)	12
4.5 Motore di simulazione degli incidenti	12
4.5.1 Anatomia di uno scenario: il caso ransomware	13
4.5.2 Coerenza tra simulazioni e motore di log/query	13
4.6 Log management e motore di query KQL	14
4.7 Sandbox di analisi statica dei file	15
4.8 SecOps AI Copilot ("Saro AI")	16
4.9 Sintesi del capitolo	16
Capitolo 5 — Analisi critica di sicurezza del progetto	18
5.1 Metodologia di analisi	18
5.2 Credenziali e password in chiaro	18
5.3 Esposizione di chiavi API lato client	18
5.4 Configurazione CORS permissiva	19
5.5 Assenza di rate limiting e protezioni anti-automazione	19
5.6 Persistenza su file system e concorrenza	19
5.7 Validazione dell'input	20
5.8 Sintesi delle criticità e piano di remediation	20
Capitolo 6 — Test e validazione	21
6.1 Strategia di test adottata	21
6.2 Casi di test rappresentativi	21
6.3 Verifica di compatibilità cross-browser	22
6.4 Limiti dell'attività di test	22
Capitolo 7 — Conclusioni e sviluppi futuri	23
7.1 Conclusioni	23
7.2 Sviluppi futuri	23
Bibliografia e sitografia	24

Appendice A — Struttura del repository di progetto	25
Appendice B — Elenco delle funzioni principali per modulo	26

Introduzione

Il presente lavoro di tesi descrive la progettazione e lo sviluppo di «SecOps Cyber Range Portal», una piattaforma web didattica pensata per riprodurre, in un ambiente controllato e privo di rischi per sistemi reali, le attività tipiche di un Security Operations Center (SOC). Il progetto è nato dall'esigenza, maturata nel corso del percorso ITS, di disporre di uno strumento pratico con cui esercitare in prima persona i concetti teorici affrontati nei moduli di sicurezza informatica: monitoraggio degli asset, gestione delle vulnerabilità, analisi dei log, risposta agli incidenti e analisi di file sospetti.

A differenza di un semplice slide-deck o di un laboratorio guidato, un cyber range interattivo permette all'utente di manipolare dati realistici, osservare in tempo reale l'evoluzione di uno scenario di attacco e mettere alla prova le proprie capacità di analisi attraverso strumenti che replicano, in forma semplificata, quelli realmente utilizzati in un SOC aziendale: dashboard di postura di sicurezza, inventario degli asset, gestione del ciclo di vita delle CVE, motore di query in stile KQL (Kusto Query Language) e un modulo di sandboxing per l'analisi statica di file eseguibili.

L'obiettivo della tesi è duplice. Da un lato, documentare il processo di analisi, progettazione e implementazione della piattaforma, illustrando le scelte architetture e tecnologiche adottate. Dall'altro, fornire una lettura critica del progetto stesso: un cyber range per la didattica della cybersecurity ha infatti la responsabilità aggiuntiva di essere, per quanto possibile, un esempio di buone pratiche — o quantomeno di rendere esplicite e consapevoli le scelte in cui, per motivi di semplicità implementativa o di ambito puramente locale/didattico, ci si è discostati da esse. Il Capitolo 5 è interamente dedicato a questa autovalutazione, condotta con lo stesso approccio metodico che si applicherebbe alla verifica di un sistema in produzione.

La trattazione è organizzata come segue. Il Capitolo 1 inquadra il contesto applicativo e didattico dei cyber range, definendo obiettivi e requisiti del progetto. Il Capitolo 2 descrive l'analisi dei requisiti e le scelte progettuali, con particolare attenzione allo stack tecnologico adottato. Il Capitolo 3 illustra l'architettura del sistema, distinguendo la componente client-side dalla componente server-side. Il Capitolo 4 costituisce il cuore tecnico dell'elaborato ed esamina in dettaglio ciascun modulo funzionale della piattaforma. Il Capitolo 5, come anticipato, presenta un'analisi critica di sicurezza del progetto stesso. Il Capitolo 6 descrive l'attività di test e validazione svolta. Il Capitolo 7, infine, trae le conclusioni e delinea i possibili sviluppi futuri.

Capitolo 1 — Contesto, obiettivi e requisiti

1.1 I cyber range come strumento didattico

Un cyber range è un ambiente simulato, isolato dalla rete di produzione, all'interno del quale è possibile riprodurre topologie di rete, asset e scenari di attacco per finalità di formazione, addestramento o test di strumenti di difesa. Nel mondo enterprise i cyber range più avanzati replicano interi segmenti infrastrutturali con macchine virtuali dedicate; in ambito didattico, specie quando l'obiettivo primario è l'apprendimento dei concetti e dei flussi di lavoro di un analista SOC, è spesso più efficace — e sostenibile in termini di risorse — adottare un approccio simulativo: dati sintetici ma realistici, scenari pre-costruiti e un'interfaccia che riproduce gli strumenti (dashboard, SIEM, sandbox) senza la necessità di infrastrutture reali sottostanti.

Questa è la scelta di fondo alla base di SecOps Cyber Range Portal: non un vero SIEM connesso a sorgenti di log reali, né una sandbox di analisi dinamica in senso stretto, ma un ambiente che ne riproduce fedelmente l'interfaccia, i flussi di lavoro e la logica di analisi, così da abbassare la barriera d'accesso per chi si avvicina per la prima volta a questi strumenti, mantenendo però aderenza terminologica e procedurale con l'operatività reale di un SOC.

1.2 Obiettivi del progetto

Il progetto si pone i seguenti obiettivi:

- fornire una dashboard di sintesi che riassume la postura di sicurezza di un'infrastruttura simulata (risk score, minacce attive, vulnerabilità aperte, uptime);
- permettere la gestione di un inventario di asset (server, workstation, dispositivi IoT) con relativo stato di sicurezza;
- gestire un catalogo di vulnerabilità in stile CVE, con severità, componente affetto e stato di remediation;
- offrire un motore di ricerca sui log in stile KQL, con regole di detection predefinite mappate su tecniche note del framework MITRE ATT&CK;
- simulare, in modo guidato e reversibile, scenari di incidente completi (ransomware, compromissione webshell, attacco alla Active Directory, DNS tunneling), con generazione di log coerenti e cronologicamente ordinati;
- offrire un modulo di analisi statica di file (calcolo hash, entropia, parsing dell'header PE, estrazione di stringhe) utile a introdurre i concetti di base della malware analysis;
- integrare un assistente conversazionale («Saro AI») in grado di rispondere a domande sul contesto corrente della piattaforma, con modalità offline di base e modalità avanzata basata su un modello linguistico esterno;
- garantire la persistenza dei dati e un sistema minimale di autenticazione multiutente.

1.3 Requisiti funzionali e non funzionali

Sulla base degli obiettivi sopra elencati sono stati definiti i seguenti requisiti.

Requisiti funzionali

ID	Requisito
RF-01	Login e registrazione di un operatore con credenziali proprie
RF-02	Visualizzazione dashboard con indicatori aggregati (risk score, minacce, vulnerabilità)
RF-03	CRUD (creazione, modifica) su asset e vulnerabilità
RF-04	Esecuzione di query predefinite sui log con evidenza dei risultati
RF-05	Avvio, avanzamento a step e reset di scenari di incidente predefiniti
RF-06	Caricamento di un file e analisi statica automatica (hash, entropia, header)
RF-07	Interazione in linguaggio naturale con il copilot Saro AI
RF-08	Persistenza dello stato applicativo tra sessioni diverse
RF-09	Cambio lingua (italiano/inglese) e tema (scuro/chiaro) dell'interfaccia

Requisiti non funzionali

ID	Requisito
RNF-01	Portabilità: funzionamento su qualunque browser moderno, senza installazioni lato client
RNF-02	Leggerezza: nessuna dipendenza da framework pesanti; avvio immediato tramite apertura di un file HTML
RNF-03	Usabilità: interfaccia coerente con l'estetica "SOC/cyberpunk" per favorire l'immersione didattica
RNF-04	Estendibilità: possibilità di aggiungere nuovi scenari, regole di detection o asset senza modificare la logica applicativa
RNF-05	Bilinguismo: interfaccia interamente localizzata in italiano e inglese

Va precisato sin da ora che i requisiti non funzionali non includono, deliberatamente, requisiti di sicurezza "production-grade" (ad es. hardening dell'autenticazione, cifratura delle credenziali, gestione sicura dei segreti): la piattaforma è concepita per un uso locale e didattico, con dati fittizi, e questa scelta è discussa e motivata approfonditamente nel Capitolo 5.

Capitolo 2 — Analisi e scelte progettuali

2.1 Analisi dello stato dell'arte

Prima di avviare lo sviluppo è stata condotta un'analisi delle soluzioni esistenti nell'ambito dei cyber range e degli strumenti di security monitoring, al fine di individuare gli elementi di interfaccia e di flusso operativo da riprendere. In particolare sono stati presi come riferimento concettuale:

- le dashboard dei SIEM enterprise (es. Microsoft Sentinel, Splunk), da cui è derivata l'idea del pannello di sintesi con risk score e la sintassi delle query in stile KQL;
- le piattaforme di analisi malware online (es. servizi di sandboxing e threat intelligence), da cui è derivata la struttura del report di analisi file (hash, verdict, indicatori);
- il framework MITRE ATT&CK, utilizzato come tassonomia di riferimento per etichettare le tecniche di attacco simulate e le relative regole di detection;
- piattaforme di cyber range accademiche e CTF (Capture The Flag), da cui è derivato l'approccio “scenario guidato a step” per le simulazioni di incidente.

Questa fase di analisi ha permesso di stabilire un principio guida per l'intero progetto: privilegiare la fedeltà concettuale e terminologica rispetto agli strumenti reali, accettando semplificazioni implementative laddove la complessità di una simulazione perfetta non avrebbe aggiunto valore didattico proporzionato.

2.2 Scelta dello stack tecnologico

La scelta dello stack tecnologico è stata guidata dai requisiti non funzionali di leggerezza e portabilità (RNF-01, RNF-02). Si è pertanto optato per un'architettura basata su tecnologie web “vanilla”, senza framework front-end (React, Vue, Angular), e un backend minimale in Node.js limitato alle sole funzionalità che richiedono necessariamente un'esecuzione server-side.

Livello	Tecnologia	Motivazione
Markup e struttura	HTML5	Nessuna dipendenza da build tool; apertura diretta del file index.html
Stile	CSS3 (custom properties)	Tema scuro/chiaro gestito tramite variabili CSS; nessun preprocessore necessario
Logica applicativa	JavaScript ES6+ (vanilla)	Massima portabilità; nessuna fase di compilazione/transpiling
Persistenza client	Web Storage API (localStorage)	Persistenza immediata senza backend per l'uso base della piattaforma
Backend opzionale	Node.js + Express	Proxy CORS e persistenza server-side per funzionalità avanzate (sandbox dinamica, multiutente)
Formato dati	JSON	Interoperabilità nativa con JavaScript; facilità di ispezione e versionamento

Questa scelta comporta un compromesso esplicito: rinunciare ai vantaggi di un framework component-based (gestione dichiarativa dello stato, componentizzazione, tooling maturo) in cambio di un'applicazione distribuibile come singolo pacchetto di file statici, eseguibile anche offline con un semplice doppio clic. Per un progetto con finalità dimostrative e didattiche, distribuito potenzialmente a più studenti su macchine eterogenee, questo compromesso è stato giudicato preferibile.

2.3 Modello dei dati

Il modello dei dati è centrato su un unico oggetto JavaScript globale, denominato DATA, inizializzato a partire da un template statico (data.js) e successivamente mantenuto sincronizzato con il Web Storage del browser. Le entità principali sono riassunte nella tabella seguente.

Entità	Descrizione	Attributi principali
stats	Indicatori aggregati di postura di sicurezza	riskScore, activeThreats, vulnCount, uptime, scoreHistory
assets	Inventario dei dispositivi dell'infrastruttura simulata	id, type, ip, os, status
vulns	Catalogo delle vulnerabilità rilevate	cve, severity, asset, status
events	Log di sicurezza generati dal sistema o dagli scenari	time, sev, msg, src, details
sandboxHistory	Cronologia delle analisi file eseguite	fileName, hash, verdict, entropy, timestamp
users	Credenziali degli operatori registrati	user, pass

La scelta di un unico oggetto in memoria, serializzato periodicamente su localStorage tramite la funzione saveData(), semplifica notevolmente la gestione dello stato in assenza di un framework reattivo, al costo di richiedere una disciplina manuale nel richiamare le funzioni di rendering dopo ogni mutazione dei dati.

2.4 Metodologia di sviluppo

Lo sviluppo è stato condotto secondo un approccio iterativo e incrementale, per moduli funzionali indipendenti (dashboard, asset, vulnerabilità, log, simulazioni, sandbox, copilot), ciascuno sviluppato e verificato singolarmente prima dell'integrazione nella shell applicativa comune. Questo approccio, mutuato dai principi Agile pur senza l'adozione formale di un framework come Scrum, ha permesso di validare progressivamente le singole funzionalità e di individuare tempestivamente le interdipendenze tra moduli (ad esempio tra il motore di simulazione e il motore di query sui log, che condividono la stessa struttura di eventi).

Capitolo 3 — Architettura del sistema

3.1 Vista d'insieme

L'architettura della piattaforma è organizzata su due livelli distinti e disaccoppiati: un'applicazione client-side, autosufficiente e responsabile della quasi totalità della logica applicativa, e un backend server-side opzionale, attivabile solo per le funzionalità che richiedono operazioni non eseguibili in sicurezza dal browser (in particolare l'inoltro di richieste verso API esterne, per aggirare le restrizioni CORS, e la persistenza multiutente su file system).

Lo schema seguente riassume il flusso dei dati tra i componenti principali.

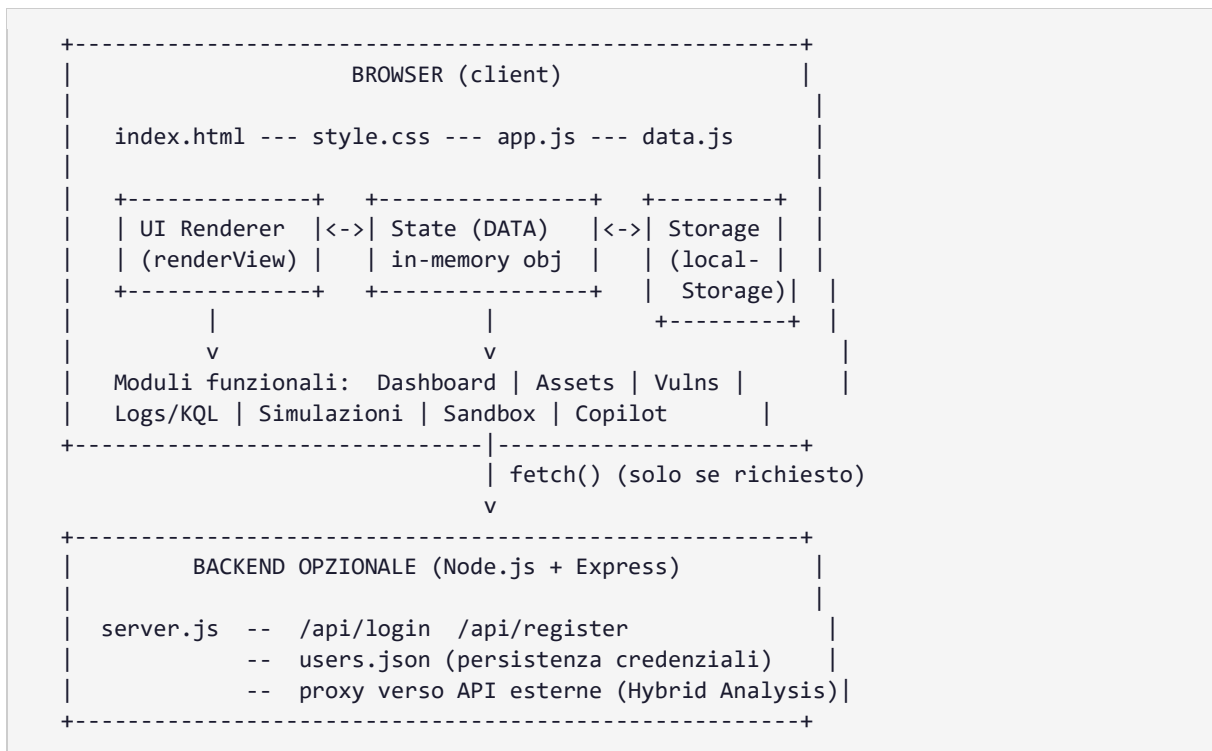


Figura 3.1 — Schema architetturale a due livelli della piattaforma

3.2 Il livello client

Il livello client è composto da quattro file principali:

- index.html — struttura semantica della pagina (shell applicativa, contenitori delle viste, modali);
- style.css — oltre 2.500 righe di stile, organizzate per componente, con supporto a tema scuro/chiaro tramite CSS custom properties;
- data.js — dataset iniziale (utenti di default, asset, vulnerabilità, eventi, regole SIEM);
- app.js — oltre 9.000 righe di JavaScript che implementano l'intera logica applicativa: routing tra viste, rendering dinamico, motore di simulazione, motore di query, analisi file, gestione audio e localizzazione.

La navigazione tra le sezioni della piattaforma (dashboard, asset, vulnerabilità, intelligence, sandbox, log, simulazione, impostazioni) è gestita da un router client-side minimale, privo di dipendenze esterne: la funzione `setupNavigation()` associa un listener a ciascuna voce del menu laterale, mentre `renderView()` si occupa di nascondere la vista corrente, invocare la funzione di rendering della vista richiesta e aggiornare lo stato attivo del menu, secondo un pattern a switch-case.

3.3 Il livello server

Il backend, implementato in `server.js`, è un'applicazione Express estremamente compatta (meno di 90 righe) che espone due sole rotte REST:

Metodo	Endpoint	Funzione
POST	/api/login	Verifica le credenziali contro il file <code>users.json</code> e restituisce l'esito
POST	/api/register	Crea un nuovo utente, verificando l'assenza di duplicati

La persistenza è realizzata tramite lettura/scrittura diretta di un file JSON (`users.json`) sul file system del server, senza l'impiego di un database relazionale o documentale. Si tratta di una scelta coerente con la scala del progetto (poche decine di utenti al più, in un contesto di laboratorio didattico) ma che, come discusso nel Capitolo 5, non sarebbe adeguata in un contesto di produzione per ragioni sia di concorrenza sia di sicurezza.

Il server svolge inoltre un secondo ruolo, quello di proxy verso l'API esterna "Hybrid Analysis" per la sandbox dinamica: poiché le policy CORS dei browser impediscono di norma a una pagina servita localmente di invocare direttamente API di terze parti che non espongono gli opportuni header, il server locale inoltra la richiesta lato server, dove tali restrizioni non si applicano.

3.4 Flusso di avvio dell'applicazione

All'apertura della pagina, la funzione `init()` coordina la sequenza di bootstrap dell'applicazione, riassunta nei passi seguenti:

1. caricamento delle preferenze utente salvate (lingua, tema, stato dell'audio) tramite `setLanguage()` e la lettura delle relative chiavi di `localStorage`;
2. caricamento del database applicativo tramite `loadLocalDatabase()`, che verifica la presenza di uno stato salvato in `localStorage` e, in sua assenza, inizializza l'applicazione con il dataset di default definito in `data.js`;
3. aggiornamento dei contenuti statici multilingua tramite `updateStaticContent()`;
4. configurazione della navigazione tramite `setupNavigation()` e avvio dell'orologio di sistema tramite `startClock()`;
5. rendering della vista di dashboard come schermata iniziale.

Questo flusso garantisce che, indipendentemente dal numero di sessioni precedenti, l'utente ritrovi lo stato esatto in cui aveva lasciato la piattaforma (asset modificati, vulnerabilità aggiornate, cronologia

della sandbox), fatta salva la possibilità di ripristinare il dataset di fabbrica tramite la funzione `resetData()` esposta nel pannello impostazioni.

Capitolo 4 — Implementazione dei moduli funzionali

Questo capitolo descrive in dettaglio l'implementazione di ciascun modulo funzionale della piattaforma, evidenziando le scelte tecniche più significative e riportando, ove utile alla comprensione, estratti di codice commentati.

4.1 Autenticazione e gestione utenti

Il modulo di autenticazione gestisce due modalità di funzionamento, corrispondenti ai due livelli architetturali descritti nel Capitolo 3: una modalità “stand-alone”, in cui le credenziali sono verificate contro l'elenco utenti mantenuto in localStorage, e una modalità “con backend”, in cui la verifica avviene tramite chiamata alle rotte /api/login e /api/register esposte da server.js.

Le funzioni login() e register() implementano rispettivamente l'accesso e la creazione di un nuovo operatore. La funzione toggleAuthMode() consente di alternare tra il modulo di login e quello di registrazione all'interno della medesima schermata. Una volta autenticato, l'operatore può cambiare la propria password tramite changePassword(), oppure disconnettersi tramite requestLogout(), che invalida la sessione corrente riportando l'utente alla schermata di accesso.

Il set di utenti precaricati nel dataset di default (data.js) comprende l'account “Analyst_01”, pensato come profilo dimostrativo standard per l'avvio rapido della piattaforma. Le implicazioni di sicurezza di questo meccanismo di autenticazione — in particolare l'assenza di hashing delle password e la password di default facilmente indovinabile — sono discusse criticamente nel Capitolo 5.

4.2 Dashboard e Security Posture Score

La dashboard costituisce la schermata di atterraggio della piattaforma e ha la funzione di offrire, in un unico colpo d'occhio, lo stato di sicurezza complessivo dell'infrastruttura simulata. La funzione renderDashboard() — la più estesa dell'intera codebase, con oltre 750 righe — si occupa di:

- calcolare e visualizzare il Security Posture Score, un indice sintetico a partire dal numero di vulnerabilità aperte, dalla loro severità e dal numero di minacce attive;
- renderizzare uno storico dell'andamento del punteggio (scoreHistory) sotto forma di grafico a linee costruito con primitive SVG native, senza librerie di charting esterne;
- mostrare una mappa di rete interattiva (Cyber Threat Map) con i nodi degli asset principali e tooltip contestuali al passaggio del mouse;
- elencare gli eventi di sicurezza più recenti tramite renderEventRows(), con codifica colore per severità (INFO, WARN, CRITICAL);
- esporre i controlli di avvio rapido per le simulazioni di incidente e per il copilot Saro AI.

Il calcolo del punteggio di postura è incapsulato nella funzione updateSecurityPostureScore(), invocata ogni volta che lo stato applicativo cambia in modo rilevante (ad esempio alla chiusura di una vulnerabilità o all'avvio di uno scenario di attacco), così da mantenere l'indicatore sempre coerente

con lo stato corrente dei dati, secondo una logica reattiva realizzata manualmente in assenza di un framework che la fornisca nativamente.

4.3 Gestione degli asset

Il modulo asset (`renderAssets()`) presenta l'inventario dei dispositivi dell'infrastruttura simulata in forma tabellare, con possibilità di filtro per tipologia (server, workstation, database, dispositivi IoT) e per stato di sicurezza (secure, warning, critical). Ogni asset è caratterizzato da un identificativo (hostname), indirizzo IP, sistema operativo e stato corrente.

La funzione `editAsset(id)` apre un modulo di modifica (`renderAssetForm()`) che consente di aggiornare i metadati dell'asset selezionato; il salvataggio avviene tramite `saveAsset()`, che aggiorna l'oggetto `DATA.assets` e invoca `saveData()` per la persistenza. Lo stato di un asset può inoltre essere modificato programmaticamente dal motore di simulazione, ad esempio per riflettere la compromissione di una workstation nel corso di uno scenario di ransomware (cfr. §4.5).

4.4 Gestione delle vulnerabilità (CVE)

Il modulo vulnerabilità (`renderVulns()`) riproduce un elenco di CVE realistiche, ciascuna associata a un asset dell'inventario, una severità (secondo la scala CVSS qualitativa: Low, Medium, High, Critical) e uno stato di remediation (Open, In Progress, Patched). Le funzioni `editVuln(cve)`, `renderVulnForm()` e `saveVuln()` replicano, per questa entità, lo stesso pattern CRUD già descritto per gli asset.

Questo modulo ha una finalità didattica specifica: introdurre il concetto di vulnerability management come processo ciclico (identificazione, valutazione, remediation, verifica) piuttosto che come semplice elenco statico, incoraggiando l'operatore a ragionare sulla priorità di intervento in funzione della severità e della criticità dell'asset coinvolto.

4.5 Motore di simulazione degli incidenti

Il motore di simulazione è il modulo più articolato della piattaforma e ne rappresenta il cuore didattico: consente di avviare uno scenario di attacco predefinito e di osservarne l'evoluzione passo dopo passo, con generazione automatica di log coerenti, aggiornamento dello stato degli asset coinvolti e incremento degli indicatori di rischio in dashboard. Sono implementati quattro scenari, ciascuno gestito da una funzione dedicata:

Scenario	Funzione	Tecniche MITRE ATT&CK simulate
Ransomware	<code>triggerRansomwareSimulation()</code>	T1566 Phishing, T1204 User Execution, T1562 Impair Defenses, T1486 Data Encrypted for Impact
Webshell	<code>triggerWebshellSimulation()</code>	T1190 Exploit Public-Facing Application, T1505.003 Web Shell, T1059 Command and Scripting Interpreter

Scenario	Funzione	Tecniche MITRE ATT&CK simulate
Attacco Active Directory	triggerADSimulation()	T1110 Brute Force, T1558 Steal or Forge Kerberos Tickets (Golden/Silver Ticket), T1003 OS Credential Dumping
DNS Tunneling	triggerDNSTunnelingSimulation()	T1071.004 Application Layer Protocol: DNS, T1048 Exfiltration Over Alternative Protocol

L'ancoraggio esplicito di ogni fase dello scenario a una tecnica del framework MITRE ATT&CK non è un dettaglio estetico: è la scelta che trasforma la simulazione da semplice “effetto scenico” a strumento didattico strutturato, perché costringe l'operatore ad associare ogni evento osservato a una categoria di comportamento avversario riconosciuta e documentata, esattamente come avviene nell'attività di un analista SOC reale in fase di triage.

4.5.1 Anatomia di uno scenario: il caso ransomware

Per illustrare il funzionamento del motore si prende ad esempio lo scenario ransomware, il più completo tra quelli implementati. All'avvio, triggerRansomwareSimulation() esegue le seguenti operazioni in sequenza:

6. aggiorna gli indicatori aggregati in DATA.stats (numero di minacce attive, scenario attivo corrente);
7. marca come critico l'asset bersaglio (la workstation “WS-HR-004.corp.internal”), simulando la compromissione;
8. genera una sequenza di eventi di log con timestamp relativi coerenti (calcolati come offset rispetto all'istante di avvio), a partire dalla ricezione di un'email di phishing con allegato a doppia estensione (invoice_copy.pdf.exe), fino all'esecuzione del payload, alla disabilitazione dei servizi di protezione e alla cifratura dei file;
9. ciascun evento include un campo details strutturato in formato coerente con i log reali di sistema (ad es. Event ID 4688 di Windows per la creazione di un processo, con campi ParentProcessName, NewProcessName, CommandLine, FileHash_SHA256), così da abituare l'operatore alla lettura di log realistici e non semplificati oltre il necessario.

Il seguente estratto (semplificato) mostra la struttura di un singolo evento generato dallo scenario:

```
{
  time: "14:32:07",
  sev: "CRITICAL",
  msg: "Processo avviato: ...\\invoice_copy.pdf.exe (PID: 8432)",
  src: "WS-HR-004.corp.internal",
  details: {
    EventID: 4688,
    ParentProcessName: "...\\OUTLOOK.EXE",
    NewProcessName: "...\\invoice_copy.pdf.exe",
    CommandLine: "\"invoice_copy.pdf.exe\" --silent",
    FileHash_SHA256: "a9f87c5e..."
  }
}
```

Figura 4.1 — Struttura di un evento di log generato dallo scenario ransomware

L'avanzamento tra le fasi dello scenario è gestito da `updateTimelineStepsDuringSimulation()`, che aggiorna una timeline grafica mostrando all'operatore in quale fase della kill chain si trova l'attacco simulato (accesso iniziale, esecuzione, evasione delle difese, impatto). Al termine, o su richiesta esplicita dell'operatore, `resetRansomwareSimulation()` ripristina lo stato originale di asset e indicatori, rendendo l'esercitazione ripetibile.

4.5.2 Coerenza tra simulazioni e motore di log/query

Un aspetto architetturale rilevante è che gli eventi generati dalle simulazioni vengono inseriti nello stesso array `DATA.events` consultato dal modulo log e dal motore di query KQL (§4.6). Questo significa che, dopo l'avvio di uno scenario, l'operatore può passare alla sezione Log e utilizzare le regole di detection predefinite per “cacciare” effettivamente gli indicatori dell'attacco appena simulato, chiudendo il cerchio tra generazione dell'incidente e sua identificazione — la stessa dinamica, in scala ridotta, di un esercizio di tipo “purple team”.

4.6 Log management e motore di query KQL

Il modulo Log (`renderLogs()`, `renderLogsRows()`, `filterLogs()`) presenta l'intero storico degli eventi generati dalla piattaforma — sia quelli di baseline sia quelli prodotti dalle simulazioni — in una tabella filtrabile per severità, sorgente e testo libero, con evidenziazione sintattica del testo tramite `highlightKQL()`.

Il cuore didattico del modulo è tuttavia il motore di query in stile KQL, esposto dalla funzione `window.runKQLQuery()`. Anziché implementare un parser completo del linguaggio Kusto — operazione fuori scopo per un progetto didattico di questa natura — la piattaforma associa a ciascuna delle venti regole SIEM predefinite (`SIEM_RULES`, definite in `data.js`) una funzione di filtro JavaScript equivalente, mostrando però all'operatore la query KQL corrispondente in linguaggio naturale/sintassi Kusto. Questa scelta consente di insegnare la logica del query language — filtri su campi, operatori di confronto, ricerca testuale — senza il costo implementativo di un motore di parsing generico.

A titolo di esempio, la regola `RULE-005`, associata alla rilevazione della tecnica `T1486` (Data Encrypted for Impact), filtra gli eventi il cui campo `details.RuleName` corrisponde esattamente a tale tecnica; la regola `RULE-018` individua invece pattern di DNS beaconing verso un dominio di comando e controllo simulato. La tabella seguente riporta un estratto delle regole implementate.

ID	Descrizione	Tecnica ATT&CK
RULE-001	Allegato con doppia estensione eseguibile	T1566.001 Spearphishing Attachment
RULE-003	Comando di arresto del servizio antivirus	T1562.001 Disable or Modify Tools
RULE-005	Evento di cifratura dati per impatto	T1486 Data Encrypted for Impact
RULE-008	Pattern di SQL Injection nella query string	T1190 Exploit Public-Facing Application
RULE-013	Fallimenti di autenticazione ripetuti (Event ID 4625)	T1110 Brute Force

ID	Descrizione	Tecnica ATT&CK
RULE-015	Ticket Kerberos con cifratura RC4 (indicatore Golden Ticket)	T1558.001 Golden Ticket
RULE-018	Pattern di DNS beaconing verso dominio C2	T1071.004 Application Layer Protocol: DNS
RULE-019	Volume anomalo di query DNS (tunneling)	T1048 Exfiltration Over Alternative Protocol

Il risultato di ogni query è presentato in una tabella con evidenza del numero di righe corrispondenti, coerentemente con l'esperienza utente di un motore di query reale, e con feedback sonoro differenziato (successo/nessun risultato) tramite il sotto-sistema audio descritto in §4.8.

4.7 Sandbox di analisi statica dei file

Il modulo sandbox costituisce l'introduzione della piattaforma ai concetti di base della malware analysis. A differenza degli altri moduli, che lavorano su dati sintetici precaricati, la sandbox elabora effettivamente, lato client, i byte del file caricato dall'operatore, tramite l'API FileReader e la manipolazione di oggetti Uint8Array. Le funzioni principali sono:

- `handleFileSelect(input)` — gestisce la selezione del file e ne legge il contenuto binario;
- `calculateEntropy(u8Array)` — calcola l'entropia di Shannon del file (o di una sua sezione), un indicatore comunemente utilizzato per rilevare la presenza di packing o cifratura: valori prossimi a 8 bit/byte segnalano dati ad alta casualità, tipici di eseguibili compressi/offuscati o di ransomware payload già cifrati;
- `parsePEHeaders(u8)` — esegue un parsing manuale, byte a byte, dell'header PE (Portable Executable) dei file Windows, estraendo architettura target, numero e nomi delle sezioni, entry point, subsystem e, per ciascuna sezione, l'entropia locale, al fine di individuare sezioni sospette (`isSuspicious` quando l'entropia di sezione supera la soglia empirica di 7,2 bit/byte);
- `extractBinaryStrings(u8)` — estrae le sequenze di caratteri stampabili dal file, utile per individuare indicatori testuali (URL, chiavi di registro, comandi) incorporati nel binario;
- `auditScriptContent(u8, fileName)` — per i file di script (batch, PowerShell, VBScript) esegue un controllo pattern-based alla ricerca di costrutti sospetti (download remoti, offuscamento, comandi di esecuzione dinamica);
- `generateHexDumpHTML()`, `generatePESectionTableHTML()`, `generateExtractedStringsHTML()`, `generateIntelCheckHTML()` — funzioni di presentazione che traducono i risultati dell'analisi in un report HTML strutturato, in stile threat-intelligence.

Il seguente estratto mostra l'implementazione del calcolo dell'entropia di Shannon, applicata sull'intero file o su una singola sezione PE:

```
function calculateEntropy(u8Array) {
  const freqs = new Array(256).fill(0);
  for (let i = 0; i < u8Array.length; i++) freqs[u8Array[i]]++;
```



```

let entropy = 0;
const len = u8Array.length;
for (let i = 0; i < 256; i++) {
  if (freqs[i] > 0) {
    const pr = freqs[i] / len;
    entropy -= pr * Math.log2(pr);
  }
}
return entropy; // valore atteso: 0 (min) - 8 (max) bit/byte
}

```

Figura 4.2 — Implementazione del calcolo dell'entropia di Shannon

Per non richiedere la disponibilità di file eseguibili reali (potenzialmente malevoli) durante le esercitazioni, la piattaforma include `createMockPEBytes()`, una funzione che genera artificialmente una sequenza di byte con struttura PE valida ma contenuto innocuo, utilizzabile per dimostrare il funzionamento del parser senza introdurre rischi. Questa scelta è coerente con il principio, già richiamato al §1.1, di privilegiare la sicurezza dell'ambiente didattico anche a costo di una minore aderenza a un caso d'uso realistico.

È infine disponibile un'integrazione opzionale (`callHAApi()`, `pollHAJobStatus()`) con il servizio esterno di sandboxing dinamico Hybrid Analysis / Falcon Sandbox, mediata dal proxy server descritto nel §3.3, che consente — quando configurata una chiave API personale — di sottoporre realmente un file all'esecuzione in una macchina virtuale cloud e osservarne i log comportamentali in tempo reale, superando così i limiti intrinseci dell'analisi puramente statica.

4.8 SecOps AI Copilot (“Saro AI”)

L'ultimo modulo funzionale è un assistente conversazionale integrato nell'interfaccia, pensato per supportare l'operatore nell'interpretazione dei dati correnti della piattaforma (asset selezionato, eventi recenti, esito di un'analisi sandbox) rispondendo a domande in linguaggio naturale. Il modulo è progettato con due modalità di funzionamento, gestite in modo trasparente per l'utente:

- modalità offline — `getOfflineResponse(userMsg, isIt)` implementa un motore di risposta a regole (pattern matching su parole chiave) che copre le domande più frequenti relative allo stato corrente della piattaforma, senza richiedere connettività né chiavi API;
- modalità avanzata — se l'operatore configura, nel pannello Impostazioni, una chiave API personale per un modello linguistico esterno (Google Gemini), le richieste vengono inoltrate al servizio remoto insieme a un contesto sintetico dello stato applicativo, costruito da `getSelectedNodeContext()`, ottenendo risposte più articolate e contestualizzate.

La funzione `setupCopilotEvents()` collega gli eventi di interfaccia (invio messaggio, selezione di suggerimenti rapidi tramite `renderSuggestions()`) alla logica di risposta, mentre `displayCopilotResponse()` implementa un effetto “macchina da scrivere” (typewriter) per la presentazione progressiva del testo, a scopo di maggiore immersività. La cronologia della conversazione è persistita in `localStorage` (`portal_copilot_history`), così da mantenere il contesto tra sessioni successive.

Dal punto di vista architetturale, questo modulo è l'unico che prevede una comunicazione diretta browser → servizio cloud di terze parti, bypassando il backend locale; le implicazioni di questa scelta in termini di gestione della chiave API sono discusse nel Capitolo 5.

4.9 Sintesi del capitolo

La tabella seguente riepiloga i moduli descritti, la relativa area di codice e la finalità didattica primaria di ciascuno, a chiusura del capitolo.

Modulo	Finalità didattica primaria
Autenticazione	Gestione delle identità e del controllo accessi
Dashboard	Lettura sintetica della postura di sicurezza (security metrics)
Asset	Asset management e superficie di attacco
Vulnerabilità	Vulnerability management e prioritizzazione del rischio
Simulazioni	Kill chain, tattiche e tecniche MITRE ATT&CK
Log / KQL	Threat hunting e query language per SIEM
Sandbox	Fondamenti di malware analysis statica
Copilot AI	Uso assistito dell'intelligenza artificiale in ambito SOC

Capitolo 5 — Analisi critica di sicurezza del progetto

Coerentemente con lo spirito di un percorso di studi in cybersecurity, questo capitolo sottopone SecOps Cyber Range Portal alla stessa disciplina di analisi che si applicherebbe a un sistema terzo: si individuano le debolezze presenti nell'implementazione attuale, se ne valuta la severità nel contesto d'uso previsto (didattico, locale, dati fittizi) e si propongono le contromisure che sarebbero necessarie qualora il progetto evolvesse verso un utilizzo multiutente reale o esposto in rete. Questo esercizio di autovalutazione è considerato parte integrante del valore formativo della tesi: costruire uno strumento per insegnare la sicurezza è anche un'occasione per praticare il pensiero critico verso il proprio stesso codice.

5.1 Metodologia di analisi

L'analisi è stata condotta tramite revisione manuale del codice sorgente (static code review), integrata da un confronto con la categorizzazione OWASP Top 10, adattata al contesto di un'applicazione web con backend minimale. Per ciascuna criticità individuata si riporta: la descrizione del problema, la sua collocazione nel codice, la severità stimata nel contesto d'uso attuale e la contromisura raccomandata.

5.2 Credenziali e password in chiaro

Il file `users.json`, così come il template `PORTAL_DEFAULT_USERS` in `data.js`, memorizza le password degli operatori in chiaro, senza alcuna funzione di hashing (es. `bcrypt`, `argon2`) né salting. La verifica delle credenziali in `server.js` confronta direttamente le stringhe in ingresso con quelle memorizzate.

Aspetto	Valutazione
Severità nel contesto attuale	Bassa — uso locale, dati fittizi, nessun dato reale a rischio
Severità in un contesto reale	Critica — compromissione totale delle credenziali in caso di accesso al file
Contromisura raccomandata	Hashing con algoritmo dedicato (<code>bcrypt/argon2</code>) e salt univoco per utente; mai confronto diretto di password in chiaro

5.3 Esposizione di chiavi API lato client

Le chiavi API per i servizi esterni (Gemini per il copilot, Hybrid Analysis e VirusTotal per la sandbox) sono acquisite tramite un campo di input nel pannello Impostazioni e memorizzate in `localStorage` (`portal_gemini_key`, `portal_ha_key`, `portal_vt_key`). Nel caso del copilot, la chiamata al servizio Gemini avviene direttamente dal browser, rendendo la chiave visibile a chiunque ispezioni il traffico di rete o il codice della pagina.

Questo pattern — segreti applicativi conservati e utilizzati interamente lato client — è una delle criticità più rilevanti individuate, ed è aggravato dal fatto che, durante la fase di sviluppo, una chiave dimostrativa è stata riportata in chiaro anche nel file `README.md` del repository, come istruzione di configurazione rapida.

Aspetto	Valutazione
Severità nel contesto attuale	Alta — una chiave pubblicata in un repository, anche a scopo dimostrativo, va considerata compromessa e va revocata
Severità in un contesto reale	Critica — uso non autorizzato del servizio a carico del titolare della chiave, possibile esaurimento di quota o costi
Contromisura raccomandata	Le chiamate a servizi terzi che richiedono un segreto vanno sempre mediate da un backend (pattern già adottato correttamente per Hybrid Analysis, cfr. §3.3, ma non per Gemini); nessuna chiave, nemmeno dimostrativa, va mai inclusa in un repository o in documentazione versionata

È opportuno segnalare che, in fase di redazione della presente tesi, la chiave dimostrativa presente nel README è stata trattata come potenzialmente esposta e non viene qui riprodotta; si raccomanda, come prima azione correttiva, la sua revoca immediata sulla console del fornitore e l'adozione sistematica di un file `.env` escluso dal controllo di versione (tramite `.gitignore`) per qualunque segreto futuro, unitamente a un backend proxy anche per le chiamate al modello linguistico, sul modello già esistente per Hybrid Analysis.

5.4 Configurazione CORS permissiva

Il backend Express abilita il middleware `cors()` senza restrizioni di origine (`app.use(cors())`), consentendo quindi richieste da qualunque dominio. In un contesto locale e didattico l'impatto è trascurabile, ma si tratta di una configurazione che andrebbe sempre esplicitata e ristretta ai soli domini attesi, per evitare che un'eventuale esposizione accidentale del servizio in rete diventi immediatamente sfruttabile da pagine di terze parti.

5.5 Assenza di rate limiting e protezioni anti-automazione

Le rotte `/api/login` e `/api/register` non implementano alcun meccanismo di limitazione della frequenza delle richieste (rate limiting), né protezioni contro tentativi automatizzati di indovinare le credenziali (brute force). In un ambiente didattico locale il rischio è nullo; qualora il backend venisse esposto pubblicamente, l'assenza di tali protezioni — unitamente alla mancanza di hashing (§5.2) — renderebbe l'endpoint di login vulnerabile ad attacchi di enumerazione e credential stuffing. Si raccomanda l'adozione di una libreria dedicata (es. `express-rate-limit`) e l'introduzione di un ritardo progressivo o di un blocco temporaneo dopo un numero configurabile di tentativi falliti.

5.6 Persistenza su file system e concorrenza

La scrittura diretta su `users.json` tramite `fs.writeFileSync()`, priva di meccanismi di locking, espone il sistema a condizioni di race condition in presenza di richieste di registrazione concorrenti: due scritture quasi simultanee potrebbero sovrasciversi a vicenda, con perdita di uno dei due nuovi utenti. Il rischio è proporzionale al numero di utenti concorrenti, verosimilmente basso in un contesto di laboratorio didattico con pochi operatori, ma è comunque un limite architetturale da tenere presente:

un'evoluzione verso un database con garanzie transazionali (anche leggero, come SQLite) risolverebbe strutturalmente il problema.

5.7 Validazione dell'input

Le rotte del backend eseguono una validazione minima dell'input (verifica di presenza dei campi user e pass), ma non impongono vincoli su lunghezza, complessità o caratteri ammessi per username e password, né eseguono sanitizzazione. Trattandosi di un file JSON scritto e letto dalla stessa applicazione (e non di una query verso un database relazionale), il rischio di injection in senso classico (SQL Injection) non è applicabile in questo componente specifico; resta tuttavia buona pratica imporre requisiti minimi di robustezza sulle password anche in ambito didattico, quale ulteriore elemento di coerenza pedagogica con i principi insegnati nel percorso di studi.

5.8 Sintesi delle criticità e piano di remediation

La tabella seguente riassume le criticità individuate secondo una scala di priorità qualitativa, coerente con l'approccio di vulnerability management già introdotto nel modulo applicativo descritto al §4.4, applicato qui — ricorsivamente — al progetto stesso.

Criticità	Priorità remediation	Sforzo stimato
Chiave API esposta in documentazione	Immediata (revoca)	Basso
Password in chiaro	Alta	Medio
Chiamate a servizi AI dirette dal client	Alta	Medio
Assenza di rate limiting sul login	Media	Basso
CORS permissivo	Media	Basso
Concorrenza su file JSON	Bassa (nel contesto attuale)	Medio-Alto

Va ribadito, a chiusura del capitolo, che nessuna delle criticità sopra descritte compromette la validità del progetto rispetto ai suoi obiettivi dichiarati: uno strumento didattico ad uso locale, con dati sintetici, non persegue — né dichiara di perseguire — gli stessi requisiti di sicurezza di un sistema esposto in produzione. La loro documentazione esplicita in questo capitolo ha lo scopo di rendere trasparente il confine tra le due categorie e di trasformare i limiti riscontrati in materiale didattico esso stesso: comprendere perché una scelta implementativa è accettabile in un contesto e pericolosa in un altro è, in ultima analisi, una delle competenze centrali che un percorso ITS in cybersecurity si propone di sviluppare.

Capitolo 6 — Test e validazione

6.1 Strategia di test adottata

Data l'assenza di un framework di testing automatizzato (scelta coerente con l'impostazione “vanilla” dello stack, cfr. §2.2), la validazione della piattaforma è stata condotta principalmente tramite test manuali strutturati, integrati da verifiche puntuali in console del browser per i moduli a maggiore complessità logica (parsing PE, calcolo dell'entropia, motore di query). Per ciascun modulo funzionale è stato definito un insieme di casi di test, eseguiti sistematicamente dopo ogni modifica significativa al codice.

6.2 Casi di test rappresentativi

Modulo	Caso di test	Esito atteso
Autenticazione	Login con credenziali di default (Analyst_01/admin)	Accesso consentito, redirect a dashboard
Autenticazione	Login con credenziali errate	Messaggio di errore, accesso negato
Autenticazione	Registrazione con username già esistente	Errore 409, nessuna duplicazione in users.json
Asset	Modifica dello stato di un asset e salvataggio	Persistenza della modifica dopo refresh della pagina
Simulazione	Avvio scenario ransomware	Comparsa eventi in sequenza cronologica corretta; asset WS-HR-004 marcato critical
Simulazione	Reset dello scenario	Ripristino dello stato iniziale di asset e indicatori
Log/KQL	Esecuzione RULE-005 dopo scenario ransomware attivo	Almeno un evento corrispondente restituito
Log/KQL	Esecuzione di una regola senza eventi corrispondenti	Messaggio “nessun risultato”, nessun errore in console
Sandbox	Upload di un file PE valido (generato con createMockPEBytes)	Parsing corretto di sezioni, entry point, subsystem
Sandbox	Calcolo entropia su file a contenuto casuale vs. testuale	Entropia prossima a 8 per il file casuale, sensibilmente inferiore per il file testuale
Copilot	Domanda in modalità offline (nessuna chiave configurata)	Risposta generata dal motore a regole, nessuna chiamata di rete
Localizzazione	Cambio lingua IT ↔ EN	Aggiornamento di tutte le stringhe statiche e dinamiche visibili
Persistenza	Chiusura e riapertura del browser	Stato applicativo (asset, vulnerabilità, cronologia sandbox) invariato

6.3 Verifica di compatibilità cross-browser

La piattaforma è stata verificata sui principali browser desktop (Google Chrome, Mozilla Firefox, Microsoft Edge), confermando il corretto funzionamento delle API utilizzate (FileReader, localStorage, Fetch API, SVG inline). Non sono state riscontrate incompatibilità significative, coerentemente con la scelta di limitare l'uso a funzionalità JavaScript standard ed evitare API sperimentali o soggette a prefissi vendor-specific.

6.4 Limiti dell'attività di test

L'assenza di test automatizzati (unit test, test di integrazione) rappresenta il limite più significativo dell'attività di validazione svolta: la correttezza delle circa 9.000 righe di app.js è stata verificata per via manuale e non è protetta da una suite di regressione. Questo limite, già individuato durante lo sviluppo, è discusso come possibile sviluppo futuro nel capitolo seguente.

Capitolo 7 — Conclusioni e sviluppi futuri

7.1 Conclusioni

Il progetto SecOps Cyber Range Portal ha permesso di applicare in un contesto realistico, seppur di natura simulativa, un ampio spettro di competenze acquisite durante il percorso ITS: dalla progettazione di un'architettura applicativa, alla gestione dello stato e della persistenza dei dati, fino alla comprensione approfondita di concetti propri della sicurezza informatica — framework MITRE ATT&CK, logica dei SIEM, fondamenti di malware analysis — necessari per poterli poi tradurre in funzionalità software coerenti e didatticamente efficaci.

Il lavoro ha inoltre offerto l'occasione di esercitare una competenza trasversale spesso sottovalutata in un percorso tecnico: la capacità di analizzare criticamente il proprio stesso operato, riconoscendone i limiti con lo stesso rigore che si applicherebbe alla valutazione di un sistema esterno. L'analisi di sicurezza condotta nel Capitolo 5, in particolare, ha reso esplicito come le medesime scelte implementative possano essere pienamente accettabili in un contesto didattico locale e, al contempo, del tutto inadeguate in un contesto di produzione — una distinzione concettuale centrale per chiunque si avvii alla professione di analista o sviluppatore in ambito cybersecurity.

7.2 Sviluppi futuri

Sulla base dei limiti individuati nei capitoli precedenti, si delineano i seguenti possibili sviluppi:

- hardening dell'autenticazione: introduzione di hashing delle password (bcrypt/argon2), rate limiting sulle rotte di login e, opzionalmente, autenticazione a due fattori;
- migrazione della persistenza da file JSON a un database leggero (es. SQLite) con garanzie transazionali, propedeutica a un utilizzo multiutente concorrente;
- estensione del proxy server a tutte le chiamate verso servizi AI esterni, eliminando l'esposizione di chiavi API lato client anche per il modulo copilot;
- introduzione di una suite di test automatizzati (unit test per le funzioni pure quali `calculateEntropy()` e `parsePEHeaders()`, test di integrazione per le rotte del backend), a copertura del limite descritto al §6.4;
- ampliamento del numero di scenari di incidente simulabili (es. attacco a supply chain, exploitation di vulnerabilità cloud) e del relativo set di regole di detection;
- introduzione di una modalità multiutente collaborativa, con ruoli differenziati (analista junior/senior, team lead) e assegnazione degli incidenti, per esercitazioni di gruppo;
- raccolta di metriche di apprendimento (tempo di risoluzione di uno scenario, percentuale di regole di detection utilizzate correttamente) a supporto di una valutazione oggettiva delle competenze acquisite dall'operatore.

Nel complesso, il progetto realizzato costituisce una base solida ed estendibile: l'architettura modulare adottata (cfr. Capitolo 3) consente di implementare la maggior parte degli sviluppi elencati senza

interventi strutturali sul codice esistente, confermando la validità delle scelte progettuali operate in fase di analisi.

Bibliografia e sitografia

MITRE ATT&CK® — Framework pubblico di tattiche e tecniche avversarie, <https://attack.mitre.org>

OWASP Foundation — OWASP Top 10 e OWASP Cheat Sheet Series, <https://owasp.org>

NIST — National Vulnerability Database (NVD) e Special Publication 800-61 “Computer Security Incident Handling Guide”, <https://nvd.nist.gov>

Microsoft Learn — Documentazione su Kusto Query Language (KQL) e Microsoft Sentinel, <https://learn.microsoft.com>

Mozilla Developer Network (MDN Web Docs) — Documentazione delle Web API utilizzate (FileReader, Web Storage API, Fetch API), <https://developer.mozilla.org>

Express.js — Documentazione ufficiale del framework, <https://expressjs.com>

Node.js Foundation — Documentazione ufficiale della piattaforma Node.js, <https://nodejs.org>

Shannon, C. E. — “A Mathematical Theory of Communication”, Bell System Technical Journal, 1948 (riferimento teorico per il calcolo dell'entropia utilizzato nel modulo sandbox)

Microsoft — “PE Format”, documentazione della specifica Portable Executable, <https://learn.microsoft.com/windows/win32/debug/pe-format>

Documentazione del materiale didattico e delle dispense del percorso ITS Cybersecurity relative ai moduli di Sicurezza delle Reti, Incident Response e Digital Forensics [da integrare con i riferimenti specifici forniti dai docenti del corso].

Appendice A — Struttura del repository di progetto

Per completezza si riporta la struttura dei file che compongono il progetto, come consegnato in allegato alla presente tesi.

```
sec_ops-main/  
|-- index.html      Struttura della pagina e shell applicativa  
|-- style.css       Fogli di stile (tema scuro/chiaro)  
|-- app.js          Logica applicativa (~9.170 righe)  
|-- data.js         Dataset di default (utenti, asset, eventi, regole SIEM)  
|-- server.js       Backend Express (login, registrazione, proxy API)  
|-- package.json    Dipendenze del backend Node.js  
|-- users.json      Persistenza credenziali (generato a runtime)  
|-- database.json   Persistenza dello stato applicativo (opzionale)  
|-- sample_data.json Dataset di esempio aggiuntivo  
|-- README.md       Istruzioni di avvio ed installazione
```

Figura A.1 — Struttura del repository di progetto

Appendice B — Elenco delle funzioni principali per modulo

A supporto della consultazione tecnica del codice sorgente, si riporta un indice delle funzioni JavaScript più rilevanti citate nel Capitolo 4, con il rispettivo modulo di appartenenza.

Funzione	Modulo
login(), register(), toggleAuthMode()	Autenticazione
renderDashboard(), updateSecurityPostureScore()	Dashboard
renderAssets(), editAsset(), saveAsset()	Asset
renderVulns(), editVuln(), saveVuln()	Vulnerabilità
triggerRansomwareSimulation(), triggerWebshellSimulation(), triggerADSimulation(), triggerDNSTunnelingSimulation()	Simulazioni
window.runKQLQuery(), renderLogs(), filterLogs()	Log / KQL
calculateEntropy(), parsePEHeaders(), extractBinaryStrings(), auditScriptContent()	Sandbox
setupCopilotEvents(), getOfflineResponse(), displayCopilotResponse()	Copilot AI
saveData(), loadLocalDatabase(), resetData()	Persistenza